بخش ششم: زمان‌بندی بلادرنگ (Real-Time Scheduling)

(بر اساس فایل pdf.6)

مقدمه: در سیستم‌های بلادرنگ (مثل کنترلر ترمز خودرو یا سیستم‌های پزشکی)، زمان همه چیز است.

- **بلادرنگ نرم (Soft):** تضمین نمی‌کند، فقط قول می‌دهد پردازه‌های مهم اولویت بالاتری داشته باشند.
- **بلادرنگ سخت (Hard):** تضمین قطعی می‌دهد که وظیفه باید تا قبل از ضرب‌الاجل تمام شود.

۱. کمینه‌سازی تأخیر (Minimizing Latency)

وقتی یک رویداد رخ می‌دهد، سیستم باید به سرعت واکنش نشان دهد. فاصله زمانی بین وقوع رویداد تا شروع سرویس‌دهی، **تأخیر رویداد (Event Latency)** نام دارد.

این تأخیر شامل دو بخش است:

تأخیر وقفه (Interrupt Latency):

- زمانی که طول می‌کشد تا CPU دستور فعلی را تمام کند، نوع وقفه را تشخیص دهد و به سراغ روتین سرویس وقفه (ISR) برود.
- **راه حل:** سیستم‌عامل‌های بلادرنگ زمان‌هایی که وقفه‌ها غیرفعال هستند را بسیار کوتاه نگه می‌دارند.

تأخیر اعزام (Dispatch Latency):

- زمانی که طول می‌کشد تا دیسپچر، پردازه فعلی را متوقف کند و پردازه بلادرنگ را روی CPU قرار دهد.
- **مرحله تضاد (Conflict Phase):**
  1. پس‌گرفتن (Preemption) پردازه فعلی.
  2. آزادسازی منابعی که توسط پردازه کم‌اهمیت اشغال شده‌اند.

۲. زمان‌بندی اولویتی (Priority Scheduling)

مهم‌ترین ویژگی سیستم بلادرنگ، پاسخگویی آنی است. بنابراین، الگوریتم زمان‌بندی باید **الویتی و پس‌گرفتنی (Preemptive Priority)** باشد.

مشخصات یک پردازه متناوب (Periodic): بسیاری از وظایف بلادرنگ به صورت دوره‌ای تکرار می‌شوند.

- **p (تناوب):** زمان تکرار وظیفه (نرخ = p/1).
- **d (ضرب‌الاجل):** زمانی که وظیفه باید تا آن لحظه تمام شده باشد.
- **t (زمان پردازش):** مدت زمان اجرای وظیفه روی CPU.

۳. الگوریتم نرخ-یکنواخت (Rate-Monotonic Scheduling)

این الگوریتم از اولویت‌های **ایستا (Static)** استفاده می‌کند.

- **قانون:** اولویت بر اساس **تناوب (Period)** تعیین می‌شود.
  - تناوب کوتاه‌تر (تکرار بیشتر) -> اولویت **بالاتر**.
  - تناوب طولانی‌تر (تکرار کمتر) -> اولویت **پایین‌تر**.

منطق: وظیفه‌ای که باید زود به زود اجرا شود، نیاز به پاسخگویی سریع‌تری دارد.

مثال شکست نرخ-یکنواخت: فرض کنید دو پردازه داریم:

- 1P: تناوب = ۵۰، زمان اجرا = ۲۵ (اولویت بالاتر چون تناوبش کمتر است)
- 2P: تناوب = ۸۰، زمان اجرا = ۳۵

اگر این‌ها را با نرخ-یکنواخت اجرا کنیم، 2P ممکن است ضرب‌الاجل خود را از دست بدهد، با اینکه CPU وقت خالی دارد.

نتیجه: این الگوریتم بهینه نیست و محدودیت بهره‌وری دارد (مثلاً برای ۲ پردازه، حداکثر تا ۸۳٪ بهره‌وری CPU را تضمین می‌کند).

۴. الگوریتم قدیمی‌ترین-ضرب‌الاجل-اول (EDF - Earliest Deadline First)

این الگوریتم از اولویت‌های **پویا (Dynamic)** استفاده می‌کند.

- **قانون:** اولویت بر اساس **ضرب‌الاجل (Deadline)** تعیین می‌شود.
  - ضرب‌الاجل نزدیک‌تر -> اولویت **بالاتر**.
  - هر بار که پردازه‌ای آماده می‌شود، سیستم چک می‌کند چه کسی ضرب‌الاجلش نزدیک‌تر است.

مزیت بزرگ: الگوریتم EDF از نظر تئوری **بهینه (Optimal)** است. یعنی می‌تواند تا **۱۰۰٪** بهره‌وری پردازنده را استفاده کند و تا زمانی که مجموع بار پردازه‌ها از ظرفیت CPU بیشتر نشود، همه به ضرب‌الاجل خود می‌رسند.

۵. زمان‌بندی سهم متناسب (Proportional Share Scheduling)

- **روش کار:** سیستم تعداد مشخصی سهم (Share) یا بلیط صادر می‌کند (مثلاً کلاً ۱۰۰ سهم).
- به هر برنامه تعدادی سهم اختصاص می‌یابد.
- **تضمین:** اگر برنامه‌ای N سهم داشته باشد، تضمین می‌شود که N درصد از زمان CPU را دریافت کند.
- **کنترل پذیرش:** اگر پردازه جدیدی بیاید و سهم بخواهد، سیستم فقط در صورتی قبول می‌کند که سهم آزاد کافی داشته باشد.

۶. زمان‌بندی بلادرنگ در استاندارد POSIX

استاندارد POSIX دو کلاس زمان‌بندی برای نخ‌های بلادرنگ تعریف می‌کند:

SCHED_FIFO:

- نخ‌ها بر اساس اولویت و به صورت "اولین ورود-اولین سرویس" اجرا می‌شوند.
- **بدون برش زمانی:** نخ تا زمانی که تمام نشود یا مسدود نشود، CPU را رها نمی‌کند (مگر اینکه نخ با اولویت بالاتر بیاید).

SCHED_RR:

- همانند FIFO است اما دارای **برش زمانی (Time Slice)** است.
- اگر چند نخ با اولویت یکسان باشند، بین آن‌ها نوبت چرخشی اجرا می‌شود.

توابع API:

- pthread_attr_setsched_policy: برای تنظیم سیاست (FIFO یا RR).
- pthread_attr_getsched_policy: برای دریافت سیاست فعلی.

پایان فصل ۵